Installing SW4: unified backends

N. Anders Petersson* Björn Sjögreen* Houjun Tang†

November 2026

Contents

1 Build requirements **1**

2 Build and install **2**
2.1 Native CPU 2
2.2 CUDA 3
2.3 Perlmutter environment 3

3 Run and validate **3**

4 Legacy Make builds **4**

5 Disclaimer **4**

1 Build requirements

The `unified-backends` branch builds one execution backend per executable: `OPENMP` for native CPU execution, `CUDA` for NVIDIA GPUs, or `HIP` for AMD GPUs. Use separate build and installation directories for each backend. There is no runtime backend switch or support for mixing CPU and GPU solver ranks in one run.

All builds need C, C++, and Fortran compilers, compatible MPI libraries, BLAS, and LAPACK. CPU execution also needs OpenMP. CMake 3.18 or newer is required; the supplied presets and HIP backend require 3.21 or newer. Use dependencies built with compatible compilers and the same MPI implementation.

*Center for Applied Scientific Computing, Lawrence Livermore National Laboratory, PO Box 808, Livermore CA 94551. This work performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under contract DE-AC52-07NA27344. This is contribution LLNL-SM-741310.

†Lawrence Berkeley National Laboratory

Feature	Additional requirements
Geographic projections	PROJ 6 or newer; configure with `USE_PROJ=ON`.
HDF5 input/output	Parallel HDF5 C and high-level libraries; `USE_HDF5=ON`. Required for receiver HDF5, Sfile/GMG materials, and HDF5 outputs. Serial HDF5 is rejected.
ZFP-compressed SSI	ZFP and H5Z-ZFP 1.1.0 or newer, plus HDF5; `USE_ZFP=ON`. Build ZFP with eight-bit bitstream words.
Random materials	Precision-matched MPI FFTW libraries; `USE_FFTW3=ON`.
CUDA or HIP solver	Matching GPU compiler and RAJA; Umpire is required with the default `SW4_USE_UMPIRE=ON`. Use the C++ standard required by those libraries (`SW4_GPU_CXX_STANDARD=20` by default).

The optional I/O libraries and FFTW are disabled by default. Enable the features required by the input deck. GPU builds use double precision; `SW4_PRECISION=single` is supported only for the native CPU solver.

2 Build and install

Run these commands from the repository root. Replace dependency paths and compiler choices with those for your site. Set `CMAKE_PREFIX_PATH` to the required dependency installation prefixes, separated by semicolons; `HDF5_ROOT` and `FFTW_ROOT` can select those libraries explicitly. Use an appropriate build concurrency for the host or allocation.

2.1 Native CPU

```
cmake -S . -B build/cpu -DCMAKE_BUILD_TYPE=Release \
  -DSW4_BACKEND=OPENMP -DSW4_BUILD_MOPT=ON \
  -DUSE_PROJ=ON -DUSE_HDF5=ON -DUSE_ZFP=ON -DUSE_FFTW3=ON \
  -DCMAKE_INSTALL_RPATH_USE_LINK_PATH=ON \
  -DCMAKE_INSTALL_PREFIX="$PWD/install/cpu"
cmake --build build/cpu --parallel 4
cmake --install build/cpu
```

The installed executables are `install/cpu/bin/sw4` and `install/cpu/bin/sw4mopt`. Omit `SW4_BUILD_MOPT=ON` if material inversion is not needed. This option is off by default. The CPU backend does not require CUDA, HIP, RAJA, or Umpire.

2.2 CUDA

Configure after loading the CUDA compiler and MPI environment. RAJA and Umpire must have CUDA enabled and support the target GPU architecture. The architecture value below is for NVIDIA A100 GPUs:

```
cmake -S . -B build/cuda -DCMAKE_BUILD_TYPE=Release \
  -DSW4_BACKEND=CUDA -DCMAKE_CUDA_ARCHITECTURES=80 \
  -DSW4_GPU_HOST_OPENMP=OFF -DSW4_BUILD_MOPT=OFF \
  -DUSE_PROJ=ON -DUSE_HDF5=ON -DUSE_ZFP=ON -DUSE_FFTW3=ON \
  -DCMAKE_INSTALL_RPATH_USE_LINK_PATH=ON \
  -DCMAKE_INSTALL_PREFIX="$PWD/install/cuda"
cmake --build build/cuda --parallel 4
cmake --install build/cuda
```

The installed forward solver is `install/cuda/bin/sw4`. CUDA host OpenMP is off by default. MPI uses staged host buffers by default (`SW4_GPU_MPI_BUFFERS=STAGED`); experimental prefetch options are off. A `sw4mopt` built with `SW4_BUILD_MOPT=ON`, including in a GPU build tree, uses the native CPU inversion implementation.

For HIP, use a fresh directory, `SW4_BACKEND=HIP`, a supported ROCm compiler (set `CMAKE_HIP_COMPILER` to `amdclang++`, not `hipcc`), matching RAJA/Umpire libraries, and the appropriate `CMAKE_HIP_ARCHITECTURES`. The receiver and geometry checks reported here have not validated HIP.

2.3 Perlmutter environment

The CPU and CUDA builds use separate prefixes under `/global/cfs/cdirs/m3354/perl/sw4.unifybackend/install`: `cpu/bin` and `cuda/bin`. Select the executable explicitly. The build environment uses GNU 13.2, Cray MPICH, parallel HDF5 1.14.3.9, and Cray FFTW; CUDA builds also load CUDA 13.2. The CUDA dependency stack uses RAJA 2026.07.0 and Umpire 2026.07.1.

Retain the matching runtime libraries when moving or launching an executable. Use the site's module environment and inspect `ldd` for missing libraries. CPU builds must also be checked on CPU nodes; a build on a GPU node alone does not establish CPU-node compatibility. Run simulations inside an allocation, with the CPU or GPU account and node type appropriate to the selected executable. Resource requests and queue limits follow current site policy.

3 Run and validate

Use the MPI launcher for the MPI library used at build time. For example, inside a suitable allocation:

```
export OMP_NUM_THREADS=1
mpiexec -n 2 install/cpu/bin/sw4 test.in
```

Replace the example input path with an existing deck and run from a directory where its inputs and output paths resolve correctly. On Slurm systems use `srun` with the allocation's process, thread, and GPU bindings. Start GPU runs with one visible GPU per MPI rank. See `SW4-UsersGuide.pdf` for input syntax and receiver metadata.

CTest is optional. Configure with `BUILD_TESTING=ON`, select `TESTING_LEVEL` and `MPI_NUM_TEST_PROCS`, then run

```
ctest --test-dir build/cpu --output-on-failure -j1
```

in an allocation that supports the configured MPI launcher. The presets disable CTest by default. The receiver regression entry points and their required executables are listed in `receiver-template-input.md`; they are separate from CTest.

Receiver validation covers bounded metadata batches, malformed input, flat/Gaussian receiver and source placement, and depth-gradient material planes. The material-gradient Taylor check uses native CPU `sw4` and `sw4mopt`, one Vp direction, and a small uniform model. These checks do not establish GPU model-level adjoints, peak selection for representative recordings, HIP correctness, or startup speedup at production scale.

4 Legacy Make builds

The Make workflow remains available with a site-specific `configs/make.inc`. Its executables and object directories differ from CMake's `build/<backend>/bin` layout. Keep Make and CMake outputs separate and use the configuration matching the selected backend and libraries. Do not reuse object files from another compiler or dependency stack.

5 Disclaimer

This document was prepared as an account of work sponsored by an agency of the United States government. Neither the United States government nor Lawrence Livermore National Security, LLC, nor any of their employees makes any warranty, expressed or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States government or Lawrence Livermore National Security, LLC. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States government or Lawrence Livermore National Security, LLC, and shall not be used for advertising or product endorsement purposes.